BVS1 – Betriebssysteme

Lecture 3 – First Process: Stop and Continue · Winter Semester 2026/2027

Stavros Kousidis

TH Köln · ICCT

From Execution to Process

Last Time: Make Execution Manageable

Lecture 2 ended with a requirement:

```
run an execution
      ↓
regain control
      ↓
continue it later
```

So the execution must become something that can be **managed**. Natural questions:

- **What must survive a pause so that the same execution can continue?**
- **How do we build that?**

CC BY-SA 4.0

A Process — Our Working Definition and Concept

A process is an execution that can be managed.

For our first process, managing means:

```
create -> run -> yield -> resume -> exit
```

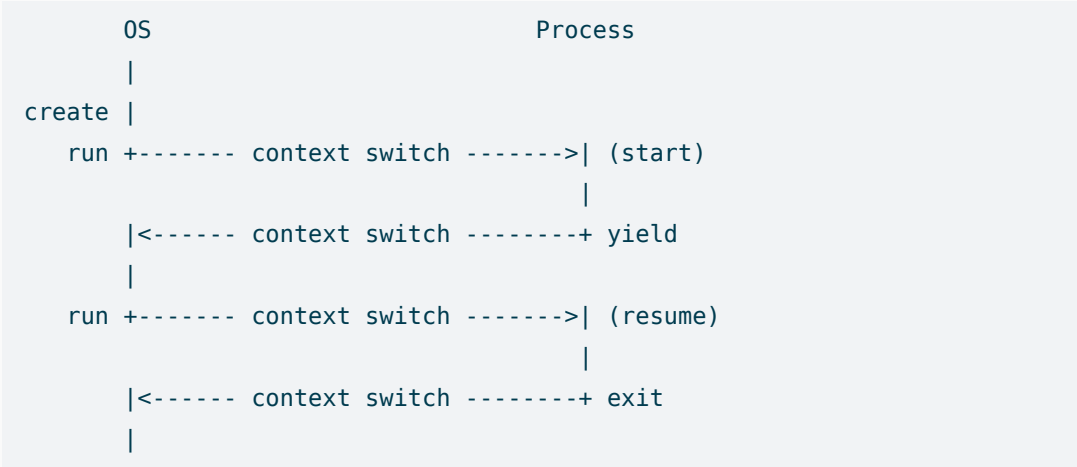
To stop and continue the **same** execution, it needs

- an entry function,
- its own stack,
- saved CPU state (context),
- and lifecycle information: state + exit status.

These are the bricks we will build for **Checkpoint 1**.

CC BY-SA 4.0

The Lifetime We Want



What does **create** do?

Creating a Runnable Process

```
create process

    prepare entry
    prepare stack
    prepare initial context
    set process state = READY
```

Creation makes a process that has never run look, to the context switch, like an execution that can be restored.

CC BY-SA 4.0

C Pointer Notation — Quick Reminder

Reference / Dereference:

```
struct process p;
struct process *q = &p;
```

`p`	the object (data structure)
`p.state`	member of object p
`&p`	address of p
`q`	pointer to p, stores &p
`*q`	object at that address (p)
`(*q).state`	member through pointer q
`q->state`	short for (*q).state

For function pointers:

```
p.entry = worker;
p.entry();
```

`worker`	address of function worker
`p.entry`	pointer to worker
`p.entry()`	call worker through entry

`worker` does not call the function.

`worker()` does.

Build the Process Object

Brick 1 — Entry Function

A process needs to remember where its execution begins.

```
void worker(void) { /* process-specific work */ }
```

Store a pointer to that function:

```
struct process {
    void (*entry)(void);
};

struct process p;
p.entry = worker; // entry points to worker
```

`worker()` can call further functions, like `helper()`, all using the same process stack.

CC BY-SA 4.0

Give the Entry Type a Name

The direct declaration works, but it is awkward to repeat:

```
void (*entry)(void);
```

Give that type a name:

```
typedef void (*process_entry_t)(void);
```

Now the struct reads more directly:

```
struct process {
    process_entry_t entry;
};
```

CC BY-SA 4.0

Brick 2 — Own Stack Storage

The process needs memory for its own call stack.

```
#define STACK_SIZE 4096

struct process {
    process_entry_t entry;
    uint8_t stack[STACK_SIZE]; // raw byte storage
};
```

For now, treat `stack` simply as **reserved memory owned by the process**.

Brick 3 — Lifecycle State and Exit Status

The manager must know whether the process may run again and, once it finishes, which exit status it produced.

```
enum process_state { READY, RUNNING, FINISHED };
```

Add that state to the object:

```
struct process {
    process_entry_t entry;
    uint8_t stack[STACK_SIZE];
    enum process_state state;
    int exit_status;
};
```

One Brick Is Still Missing

We can now remember:

```
where to start        -> entry
where calls can live  -> stack
whether it may run    -> state
how it finished       -> exit status
```

But suppose we stop here:

```
helper();
process_yield();
/* continue here later */
```

What must we preserve so execution can continue at exactly that point?

CC BY-SA 4.0

Understand the Process Stack

What Does the Stack Preserve?

Function instructions do not live on the stack. They remain in the program's `.text` section.

The stack stores **runtime data for active function calls**, for example:

```
saved return information
saved register values
some local variables
temporary values when needed
```

Not every local variable must live on the stack; the compiler may keep values in registers.

CC BY-SA 4.0

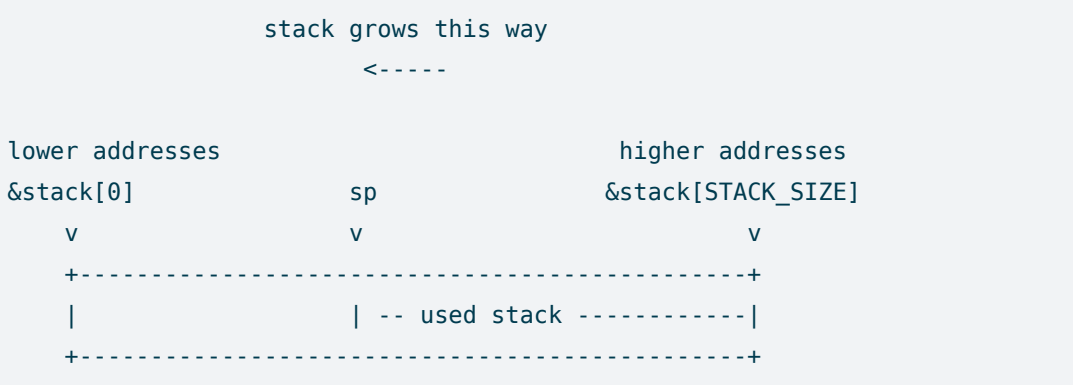
The Stack Is Memory in the Process Object

```
uint8_t stack[STACK_SIZE]; //reserved by our process
```

Conceptually:

```
lower addresses                                  higher addresses
&stack[0]                                        &stack[STACK_SIZE]
    v                                                    v
    +------------------------------------------------+
    |              reserved stack storage            |
    +------------------------------------------------+
```

On RV64 sp Selects the Active Stack Position



C array – indices increase toward higher addresses

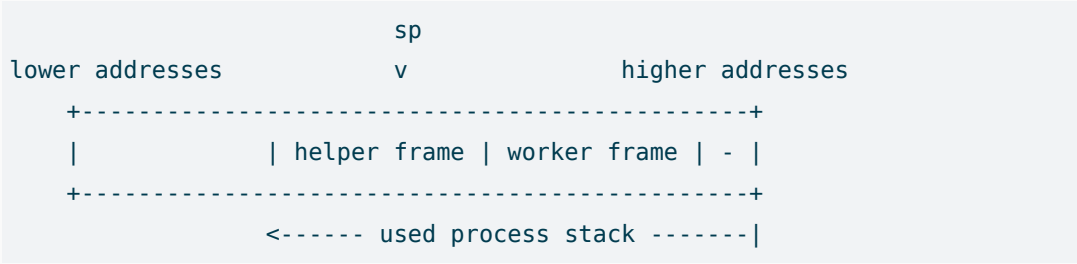
RISC-V stack (convention) – sp moves toward lower addresses

Remember: One Process, Many Function Calls

Nested calls use the **same process stack**.

```c
void helper(int *x) { (*x)++; }
void worker(void) { int n = 1; helper(&n); }
```

While `helper()` is active:



Save the Machine State

The Stack Alone Is Not Enough

At the instant we stop a process, some execution state is still in CPU registers.

For example:

```
sp   -> current stack position
ra   -> return/continuation address
s0-s11 -> values that must survive C function calls
```

If another execution uses those registers, the current values are lost.

A resumable process therefore needs saved CPU register state as well as its stack.

Brick 4 — Saved Execution Context

We call the saved CPU register state a **context**.

```
struct context {
    uintptr_t ra;    // where execution should return
    uintptr_t sp;    // where the current stack is
    uintptr_t s[12]; // preserved register-held values
};
```

Think of `context` as enough saved registers to continue this execution.

Note: `uintptr_t` is an **unsigned integer type**, not a pointer. It can hold the integer representation of an address; in our RV64 toolchain it is 64 bits wide.

CC BY-SA 4.0

The Complete Process Object

Now all four bricks fit together:

```
struct process {
    process_entry_t entry;
    uint8_t stack[STACK_SIZE];
    struct context context;
    enum process_state state;
    int exit_status;
};
```

```
entry + stack + context + lifecycle = managed execution
```

CC BY-SA 4.0

Switch Between Executions

The Low-Level Interface

The course provides one low-level function, implemented in RISC-V assembly. The following C declaration defines the interface.

```
void context_switch(
    struct context *current_context,
    const struct context *next_context);
```

Conceptually:

```
save the current CPU state into current_context
restore the next CPU state from next_context
continue the restored execution
```

CC BY-SA 4.0

context_switch() — Save the Current Context

First, the current execution is made resumable

```
context_switch:
    sd ra,   0(a0)
    sd sp,   8(a0)
    sd s0,  16(a0) // store s0 into memory at address a0 + 16 (bytes)
    ...
    sd s11,104(a0)
```

- sd stores register into memory (store doubleword).
- a0 holds the pointer to current_context (first argument register; caller-saved).
- s0 to s11 are callee-saved registers.
- a0 itself is not part of the persistent process context.

CC BY-SA 4.0

context_switch() — Restore Another Context

Then it restores another saved execution:

```
    ld ra,   0(a1)
    ld sp,   8(a1)
    ld s0,  16(a1)
    ...
    ld s11,104(a1)

    ret
```

- `ld` loads register from memory (load doubleword).
- `a1` holds the pointer to `next_context` (second argument register; caller-saved).
- `s0` to `s11` are callee-saved registers.
- `a1` itself is not restored as part of the persistent process context.

CC BY-SA 4.0

Read the Assembly as a Story

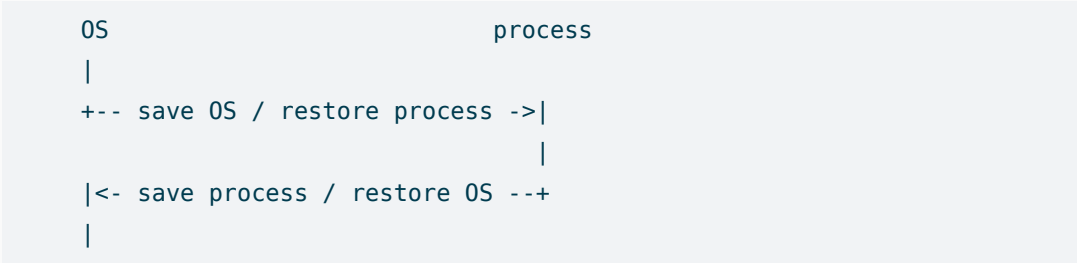
```
1. save current ra, sp, s0-s11
2. restore next ra, sp, s0-s11
3. execute ret
```

A very important part is step 3: **`ret` uses the restored `ra`**.

So `context_switch()` can return into an execution that was stopped earlier.

CC BY-SA 4.0

One Primitive, Two Directions



The same mechanism transfers control in both directions; only which context is saved and which is restored changes.

What Is Provided?

For Checkpoint 1, the architecture-specific operation `context_switch(current, next)` is provided.

You are expected to build the C-level process abstraction around it:

```
process object
create
run (resume)
yield
exit
```

Understand the assembly boundary; you do not have to reproduce it.

CC BY-SA 4.0

Build the Process API

Public Process Interface

The public C interface for our first process is:

```
void process_create(struct process *process,
                    process_entry_t entry);
void process_run(struct process *process);
void process_yield(void);
void process_exit(int status);
```

Together with the `struct process` built earlier, this is the interface used in Exercise 2 and implemented in Checkpoint 1.

CC BY-SA 4.0

process_create() — Prepare First Execution, Do Not Run

```
process_create(p, entry):
    remember entry in p
    choose aligned initial sp at the high end of p's stack
    prepare an initial context
    set p's state to READY
```

For the first run, that initial context must lead into the entry function:

```
initial ra -> internal process start -> p.entry
```

If the entry function returns, the internal process start calls `process_exit(0)`.

CC BY-SA 4.0

process_run() — Start or Resume

current_process points to the process whose execution currently owns the CPU.

```
process_run(p):
    reject p if FINISHED
    set current_process to p
    set p's state to RUNNING
    context switch from OS to p
```

The same operation covers both cases:

```
first run    -> restore prepared initial context
later run    -> restore context saved by yield
```

There is no separate process_resume().

CC BY-SA 4.0

process_yield() — Pause and Return to the OS

```
process_yield():
    set current_process state to READY
    context switch from current_process to OS
```

When the process is run again:

```
restore saved process context
          ↓
continue after process_yield()
```

Resume is restore, not restart.

`process_exit()` — Finish and Return to the OS

```
process_exit(status):
    record status
    set current_process state to FINISHED
    context switch from current_process to OS
```

Both `yield` and `exit` switch back to the OS.

```
yield -> READY     -> may run again
exit  -> FINISHED  -> must not run again
```

CC BY-SA 4.0

Put the Bricks Together

Check Your Model

What breaks if we:

1. call the entry function again after `yield`?
2. restore `ra` but not `sp` and `s0-s11`?
3. call `process_run()` after the process is `FINISHED`?

Implement your process API and play. Remember:

```
restart != resume
partial context != same execution
FINISHED != runnable
```

CC BY-SA 4.0

Key Ideas

- **Process:** a managed execution.
- **Process representation:** entry function + own stack + CPU context + lifecycle state + exit status.
- **Stack:** process-owned memory for the runtime state of active function calls.
- **Context:** saved CPU register state needed to continue the same execution.
- **Lifecycle vs. Context:** lifecycle state says **whether** an execution may run; context says **how** it continues.
- **Context switch:** save the current CPU register state, restore the next one – the same symmetric mechanism in both directions.

CC BY-SA 4.0

References

Selected references:

- RISC-V International. *RISC-V ABIs Specification* — calling convention and register roles.
- RISC-V International. *The RISC-V Instruction Set Manual, Volume I* — loads, stores, and `ret` background.
- Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces* — process abstraction and context switching.

CC BY-SA 4.0